



SensPro™

FreeRTOS Reference Guide

Rev. 20.0.0

June 2020

Documentation Control

History Table

Version	Date	Description
20.0.0	7 June 2020	Initial release, based on the CEVA-XM6/CEVA-BX FreeRTOS

Disclaimer and Proprietary Information Notice

The information contained in this document is subject to change without notice and does not represent a commitment on any part of CEVA®, Inc. CEVA®, Inc. and its subsidiaries make no warranty of any kind with regard to this material, including, but not limited to implied warranties of merchantability and fitness for a particular purpose whether arising out of law, custom, conduct or otherwise.

While the information contained herein is assumed to be accurate, CEVA®, Inc. assumes no responsibility for any errors or omissions contained herein, and assumes no liability for special, direct, indirect or consequential damage, losses, costs, charges, claims, demands, fees or expenses, of any nature or kind, which are incurred in connection with the furnishing, performance, or use of this material.

This document contains proprietary information, which is protected by U.S. and international copyright laws. All rights reserved. No part of this document may be reproduced, photocopied, or translated into another language without the prior written consent of CEVA®, Inc.

CDNN™, CEVA-Bluetooth™, CEVA-BX1™, CEVA-BX2™, CEVA-Dragonfly™, CEVA-Hillcrest Labs Freespace®, CEVA-Hillcrest Labs MotionEngine™, CEVA-MM3000™, CEVA-MM3100™, CEVA-MM3101™, CEVA-Quark™, CEVA-RivieraWaves Bluetooth™, CEVA-RivieraWaves Wi-Fi™, CEVA-SAS™, CEVA-SATA™, CEVA-Teak®, CEVA-TeakLite®, CEVA-TeakLite-II™, CEVA-TeakLite-III™, CEVA-TeakLite-4™, CEVA-TL3210™, CEVA-TL3211™, CEVA-TL410™, CEVA-TL411™, CEVA-TL420™, CEVA-TL421™, CEVA-TLS100™, CEVA-Toolbox™, CEVA-X™, CEVA-X1™, CEVA-X2™, CEVA-XC5™, CEVA-XC12™, CEVA-XC16™, CEVA-X1620™, CEVA-X1622™, CEVA-X1641™, CEVA-X1643™, CEVA-XC321™, CEVA-XC323™, CEVA-XC4210™, CEVA-XC4500™, CEVA-XM4™, CEVA-XM6™, CEVA-Xtend™, ClearVox™, NeuPro™, NeuPro-S™, PentaG™, SenslinQ™, SensPro™, and WhisPro™ are trademarks of CEVA®, Inc. and/or its subsidiaries.

All other product names are trademarks or registered trademarks of their respective owners.

Support

CEVA® makes great efforts to provide a user-friendly software and hardware development environment. Along with this, CEVA provides comprehensive documentation, enabling users to learn and develop applications on their own. Due to the complexities involved in the development of DSP applications that might be beyond the scope of the documentation, an online Technical Support Service has been established. This service includes useful tips and provides fast and efficient help, assisting users to quickly resolve development problems.

How to Get Technical Support:

- **FAQs:** Visit our website <http://www.ceva-dsp.com> or your company's protected page on the CEVA website for the latest answers to frequently asked questions.
- **Application Notes:** Visit our website <http://www.ceva-dsp.com> or your company's protected page on the CEVA website for the latest application notes.
- **Email:** Use the CEVA central support email address ceva-support@ceva-dsp.com. Your email will be forwarded automatically to the relevant support engineers and tools developers who will provide you with the most professional support to help you resolve any problem.
- **License Keys:** Refer any license key requests or problems to sdtkeys@ceva-dsp.com. For SDT license keys installation information, see the *SDT Installation and Licensing Scheme Guide*.

Email: ceva-support@ceva-dsp.com

Visit us at: www.ceva-dsp.com

Table of Contents

1. INTRODUCTION	1
1.1 Scope	1
1.2 Audience	1
1.3 Related Documents	1
1.4 Overview	2
2. FREERTOS PORT DETAILS	3
2.1 FreeRTOS Version and Original Sources	3
2.2 Opening and Building a FreeRTOS Port	3
2.3 Modifying the FreeRTOS Library	4
2.4 Linker Mapping	4
2.5 Tick Interrupt Source, Programming, and Tick Hook	5
2.6 Library Context-Switch Performance	6
2.7 Minimum Stack Size per Task	8
2.8 Library Verification	8
3. GLOSSARY.....	9

List of Figures

Figure 2-1: Disabling Exceptions on Stack Violations	3
--	---

List of Tables

Table 4-1: Acronyms	9
---------------------------	---

1. Introduction

1.1 Scope

This document describes how to integrate and use the FreeRTOS real-time operating system into the SensPro™ code, and provides programming examples.

The FreeRTOS library supports the SensPro core, and Release and Debug library versions are supplied.

1.2 Audience

This document is intended for SensPro programmers who want to use FreeRTOS in an application.

1.3 Related Documents

The following documents are related to the information in this document:

1. *SensPro Architecture Specification Volume I*
2. *SensPro Architecture Specification Volume II*
3. *SensPro Architecture Specification Volume III (MSS)*
4. *SensPro MSS Support Library API Reference Guide*
5. *CEVA Vision Platform Release Notes*

The FreeRTOS API is described in many documents at the official site <http://www.FreeRTOS.org/>.

For example, the document at <http://www.freertos.org/Documentation/Using-the-FreeRTOS-Real-Time-Kernel-A-Practical-Guide-LPC17xx-Edition-Document-Outline.pdf> describes the RTOS API. In that document, chapters 1 through 4 are relevant for all cores, and describe the API of the various FreeRTOS services.

For more details, see:

- <http://www.freertos.org/FreeRTOS-quick-start-guide.html>
- <http://www.freertos.org/FAQHelp.html>

Note: *The formal FreeRTOS Reference Manual is protected by copyrights and should be purchased on the formal FreeRTOS site.*

It is not delivered as a part of this library package.

1.4 Overview

Implementing a real-time system on any embedded platform requires real-time management of tasks and data, including task switches according to predefined priorities, interrupts, and semaphore signaling. CEVA uses an off-the-shelf operating system named **FreeRTOS**, whose source files and other required information/documentation are available at <http://www.FreeRTOS.org/>.

For FreeRTOS to be functional on the CEVA core, certain specific code parts were implemented as required by the FreeRTOS concept. This implementation mainly relates to the way a context switch is handled on the core.

This project includes a tester code that demonstrates FreeRTOS usage.

2. FreeRTOS Port Details

The code was tested for the SensPro with the Performance-Accurate Simulation mode (PACC).

2.1 FreeRTOS Version and Original Sources

The FreeRTOS version used is **V8.2.0**.

2.2 Opening and Building a FreeRTOS Port

The FreeRTOS port is part of the infrastructure package, and is built using CMake. For details, see the *CEVA Vision Platform Release Notes*.

To run applications that use FreeRTOS functionality, do the following:

1. Import the FreeRTOS project or load the FreeRTOS binary file into the IDE.
2. In the IDE, select **Run > Exceptions**, and then disable exceptions on stack violations (Figure 2-1).

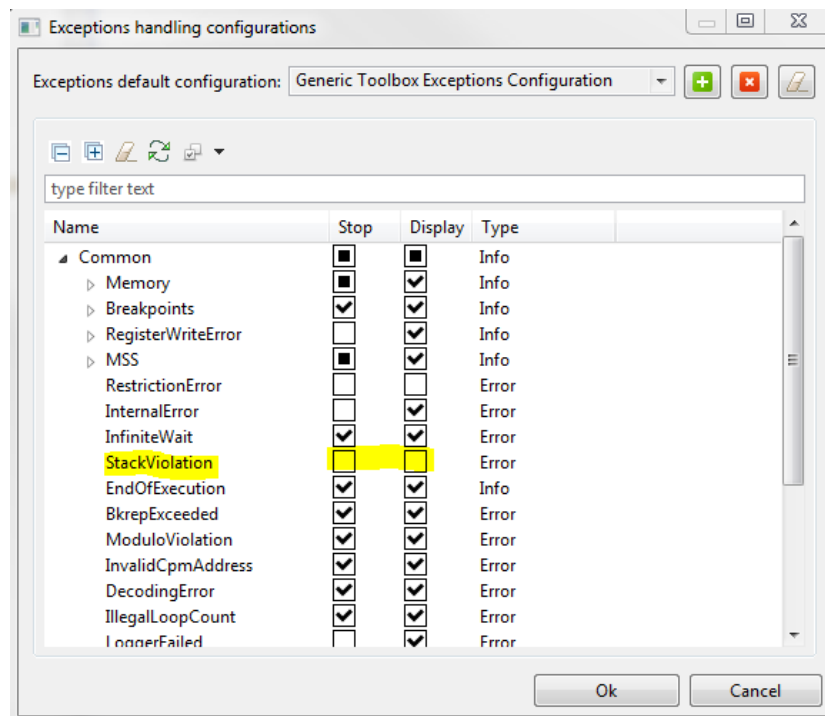


Figure 2-1: Disabling Exceptions on Stack Violations

2.3 Modifying the FreeRTOS Library

It is possible to modify the source files and use CMake to create a build. For details, see the *CEVA Vision Platform Release Notes*.

The size of the FreeRTOS code can be reduced if specific features are not used; however, they are included in the configuration. It is important to read the purpose of each flag on the FreeRTOS website (<http://www.FreeRTOS.org/>) if modifications are needed.

Important: The Heap size of the FreeRTOS is defined by the definition in the *FreeRTOSConfig.h* file:

```
//The total amount of RAM available to the
kernel.

#define configTOTAL_HEAP_SIZE      ( ( size_t )
( 10 * 1024 ) )
```

In general, heap size can be adjusted. It is based on the number of tasks/timers/queues in the system (an idle task is also defined in the configuration file, which is another task in addition to all of the user's other tasks).

2.4 Linker Mapping

Because the FreeRTOS library sections are mapped, the code/data sections can be placed in a way that best suits the project.

The following is the expected linker mapping:

- The library code is mapped to a section named **RTOS_CODE**.
- The library **uninitialized** data is mapped to a section named **RTOS_BSS**.
- The library **initialized** data is mapped to a section named **RTOS_DATA**.
- The library **constant** data is mapped to a section named **RTOS_CONST_DATA**.

If you do not assign these sections in the linker file, they will be mapped automatically by the linker.

2.5 Tick Interrupt Source, Programming, and Tick Hook

The library is currently configured to work with a **tick** interrupt source, which can be used for switching between task priorities, suspending tasks for a known time, and so on.

The tick interrupt source is usually an external hardware device (for example, a timer or an RF device) that tracks real time. The tick period is also defined in the FreeRTOS in a definition that introduces the tick period to any function inside the code, if needed. This definition is:

```
#define configTICK_RATE_HZ ( ( portTickType ) 1000 )
```

Note: When a task is suspended, for example, the suspension time is defined by system ticks and the FreeRTOS simply counts tick interrupts to know when to resume it.

The SensPro core manages the interrupts via the Interrupt Control Unit (ICU). While it is possible to configure external interrupts as the tick source, in this example, the tick source is configured via the MSS Timer. The source of the tick is currently set to be an internal interrupt that is bound to one of the MSS Timers. For details about MSS Timers and the ICU feature drivers, see the *SensPro MSS Support Library API Reference Guide*.

If external hardware is used to create the tick interrupt, it is usually required to clean the interrupt source or even to reprogram the interrupt for the next tick creation in some cases. To do this, FreeRTOS is currently configured to call the **vApplicationTickHook()** user function, which is set by the following configuration in the **FreeRTOS_config.h** file and can be modified if not needed:

```
#define configUSE_TICK_HOOK 1
```

The user is expected to program the source interrupt in the user-supplied **PlatformSetupTimerInterrupt()** function. The user can enable the interrupt assertion in this code, and FreeRTOS will unmask this interrupt only after it has started the internal scheduler and will then start counting the ticks.

It is the responsibility of the user to supply these functions; otherwise, the linking of the FreeRTOS library will fail because it is looking for these functions. An example of these hooks can be seen in the **port.c** code supplied with this project.

If the interrupt tick source is modified, the definitions here will also impact the interrupt table file that is a part of the tester environment (because the user must manage the other interrupt sources as well). The **port.asm** file should also be recompiled when the tick sources are changed.

2.6 Library Context-Switch Performance

The most important FreeRTOS feature is assumed to be context switching between tasks. FreeRTOS switches between tasks as follows:

1. Either a tick interrupt or a user switch request (which is also a type of interrupt) is received.
2. The executing task stores all of the core registers in its private stack.
3. The task calls the FreeRTOS kernel to increment the tick counter (if it is used) or call the tick hook (if it exists).
4. The FreeRTOS context-switching logic determines which task should be resumed.
5. The selected task is resumed by restoring all of the core registers that were stored by that task, as well as the program counter and stack pointer that were used by the task.

Because the SensPro core uses vector operations, the SensPro FreeRTOS port supports vector context switching. By default, vector context switching is disabled; when it is enabled, the following modes are supported:

- **Vector Context Switching:** Enables each task to store its contents on its private stack (24 vector registers are saved in the stack)
- **Extended Vector Context Switching:** Enables each task to store its contents on its private stack (all of the SensPro vector registers are saved in the stack)

To switch between these modes, make the following modifications in the source code:

- To enable Vector Context Switching mode:

1. In the **port.asm** file, change:

```
.SAVE_VECTORS_IN_CONTEXT, 0
```

to:

```
.SAVE_VECTORS_IN_CONTEXT, 1
```

2. In the **portmacro.h** file, uncomment the following line:

```
#define SAVE_VECTORS_IN_CONTEXT
```

3. Recompile the project.

- To enable Extended Vector Context Switching mode:

1. In the **port.asm** file, do the following:

- 1.a Change:

```
.SAVE_VECTORS_IN_CONTEXT, 0
```

to:

```
.SAVE_VECTORS_IN_CONTEXT, 1
```

- 1.b Change:

```
.SAVE_EXTENDED_VECTORS_IN_CONTEXT, 0
```

to:

```
.SAVE_EXTENDED_VECTORS_IN_CONTEXT, 1
```

2. In the **portmacro.h** file, uncomment the following lines:

```
#define SAVE_VECTORS_IN_CONTEXT  
#define SAVE_EXTENDED_VECTORS_IN_CONTEXT
```

3. Recompile the project.

2.7 Minimum Stack Size per Task

Because each task must be able to store its contents on its stack, it needs a minimum stack size even if the task is empty (that is, it is not running any user code that requires more stack).

This minimum stack size depends of the number of registers that are stored.

The minimum stack size for each task that is running now is set in the **FreeRTOSConfig.h** file (this size is multiplied by 4 for final size, in bytes):

```
#define configMINIMAL_STACK_SIZE( ( unsigned portSHORT ) 150 )
```

The idle task that is automatically created uses this defined size. It should be larger than the size of storing all of the registers, plus the size of any other store that might occur via any other interrupts that have a higher priority than the FreeRTOS code (they can interrupt the code at any point). This means that two store operations of the entire register set are possible, each one using 75*4 bytes:

- One store is the task context switch store.
- The other store is an interrupt that might occur inside the task code when the user enabled other interrupts in the source code (nesting interrupts are not supported).

This means that $150 * 4 = 600$ bytes of stack are allocated for the idle task stack in this default library setting. When the user creates a task, the stack size for the task is given as an argument and must be greater than the value here, and should take the user's code extra stack usage inside the specific task into consideration.

2.8 Library Verification

The library verification includes testing on the SensPro core, in both the Debug and Release configurations in the PACC simulation modes.

During compilation and linking of all configurations, no warnings or errors were found, and all tests passed verification.

3. Glossary

Table 3-1 defines the acronyms used in this document.

Table 3-1: Acronyms

Term	Definition
API	Application Program Interface
ICU	Interrupt Control Unit
PACC	Performance Accurate
RTOS	Real-Time Operating System